



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ➤ CARPENTIER
Nom d'usage ➤
Prénom ➤ Geoffrey
Adresse ➤ 498 Boulevard Jean Rostand
83500 La Seyne-Sur-Mer

Titre professionnel visé

Développeur Web et Web Mobile

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.

Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

*[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels
du ministère chargé de l'Emploi]*

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

 <http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Intitulé de l'activité-type n° 1:	p.	5
Développer la partie front-end d'une application web ou web mobile sécurisée.		
Exemple n° 1 :		
▶ Installer et configurer son environnement de travail en fonction du projet web ou web mobile	p.	5
Exemple n° 2 :		
▶ Maquetter des interfaces utilisateur statiques web ou web mobile.	p.	10
▶ Réaliser des interfaces utilisateur statiques web ou web mobile.		
Exemple n° 3 :		
▶ Développer la partie dynamique des interfaces utilisateur web ou web mobile	p.	15
Intitulé de l'activité-type n° 2:	p.	19
Développer la partie back-end d'une application web ou web mobile sécurisée		
▶ Exemple n° 1 :		
— Mettre en place une base de données relationnelle.	p.	19
▶ Exemple n° 2 :		
— Développer des composants métier côté serveur.	p.	23
▶ Exemple n° 3 :		
— Développer des composants d'accès aux données SQL et NoSQL.	p.	28
— Documenter le déploiement d'une application dynamique web ou web mobile.		
Titres, diplômes, CQP, attestations de formation (facultatif)	p.	34
Déclaration sur l'honneur	p.	35
Documents illustrant la pratique professionnelle (facultatif)	p.	36
Annexes (Si le RC le prévoit)	p.	37

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ►

Installer et configurer son environnement de travail en fonction du projet web ou web mobile (CP1)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Avant de pouvoir commencer le développement de mon projet « Livre d'or », je me suis d'abord attaché à préparer un environnement de travail fiable. J'avais besoin de disposer d'une configuration efficace afin de pouvoir coder, tester et déboguer l'application dans de bonnes conditions dès le début du projet. J'ai donc consacré une première phase à configurer mon poste sous Windows 11, qui ne disposait initialement d'aucun outil de développement.

La première étape indispensable a donc consisté à déployer un environnement d'exécution local. Mon choix s'est naturellement porté sur **Laragon** car il réunit dans une seule solution légère l'ensemble des outils dont j'avais besoin (pile WAMP) pour développer en PHP : **Apache** comme serveur HTTP, **MySQL** pour la base de données, **PHP** pour l'exécution côté serveur et **phpMyAdmin** pour l'administration de la base. Après l'installation, j'ai vérifié que chaque service fonctionnait correctement sur ses ports standards avant d'activer le HTTPS grâce au certificat auto-signé fourni par Laragon.

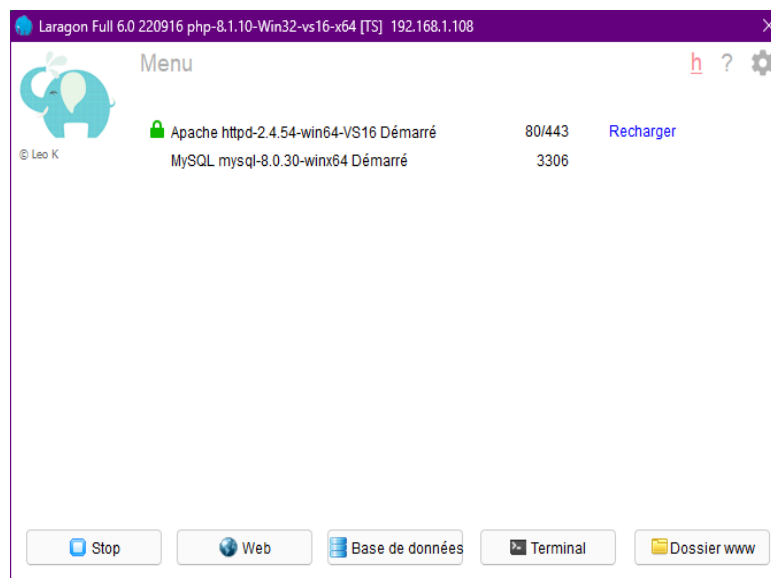


Figure 1.1 - Panneau Laragon, Apache (port 80/443) et MySQL (port 3306) démarrés.

J'ai ensuite installé et configuré mon IDE Visual Studio Code, et je lui ai ajouté quelques extensions afin de disposer d'un environnement adapté à mes besoins. L'objectif n'était pas d'accumuler les outils, mais de gagner en efficacité au quotidien : *Prettier* pour assurer un formatage homogène du code, *PHP Intelephense* pour l'auto-complétion et la détection d'erreurs, *PHP Debug* qui permet de déboguer localement, tandis que *Postman* servirait à tester les futures API. J'ai également relié VS Code à mon compte GitHub afin de retrouver rapidement mon environnement de travail sur une autre machine.

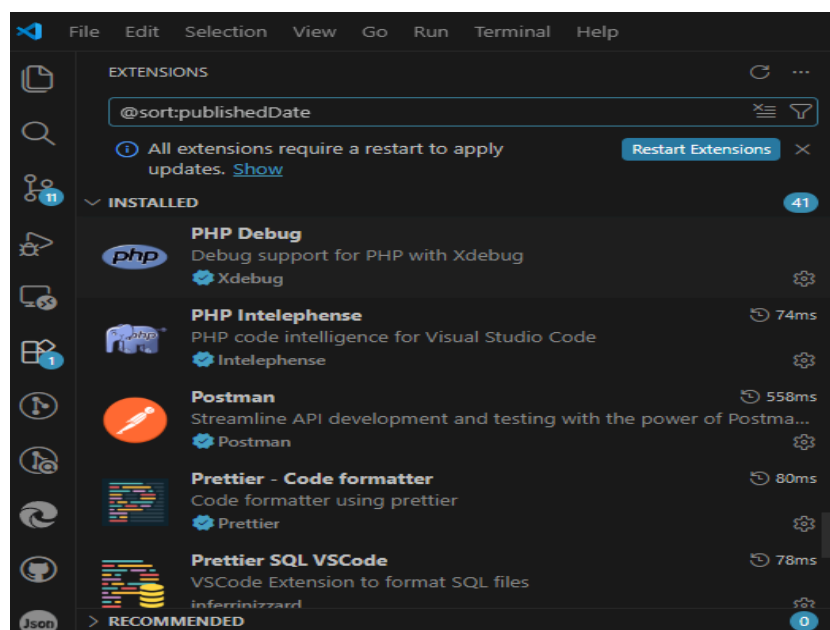


Figure 1.2 - Panneau latéral « Extensions » de VS Code

D'autre part, j'avais besoin de conserver un historique fiable de mes modifications tout au long du développement du projet. Ainsi, j'ai installé Git, avec son terminal Git Bash intégré dans VS Code, ce qui m'a permis d'utiliser directement les commandes de gestion de versions depuis mon éditeur sans devoir changer d'environnement de travail. Pour ce faire, j'ai relié mon environnement à mon identité Git en utilisant les commandes de configuration adéquates (`git config`) :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  QUERY RESULTS  POSTMAN CONSOLE

GCA@LGRam MINGW64 /d/TOOLS/LARAGON/www/livre-or
$ git config --global user.name "geoffrey-carpentier"

GCA@LGRam MINGW64 /d/TOOLS/LARAGON/www/livre-or
$ git config --global user.email "geoffrey.carpentier@laplateforme.io"
```

Figure 1.3 - Configuration de l'identité Git en ligne de commande (`git config user.name / user.email`).

Les premiers tests réalisés au sein de VS Code ont rapidement mis en évidence un problème : la commande ``php -v``, exécutée depuis le terminal de VS Code, renvoyait une erreur : le système ne parvenait pas à localiser l'exécutable PHP. J'ai pu résoudre ce problème en ajoutant le chemin d'installation à la variable d'environnement Path. Cette manipulation a permis de rétablir la communication entre le terminal et mes services locaux, en rendant PHP accessible depuis n'importe quel dossier.

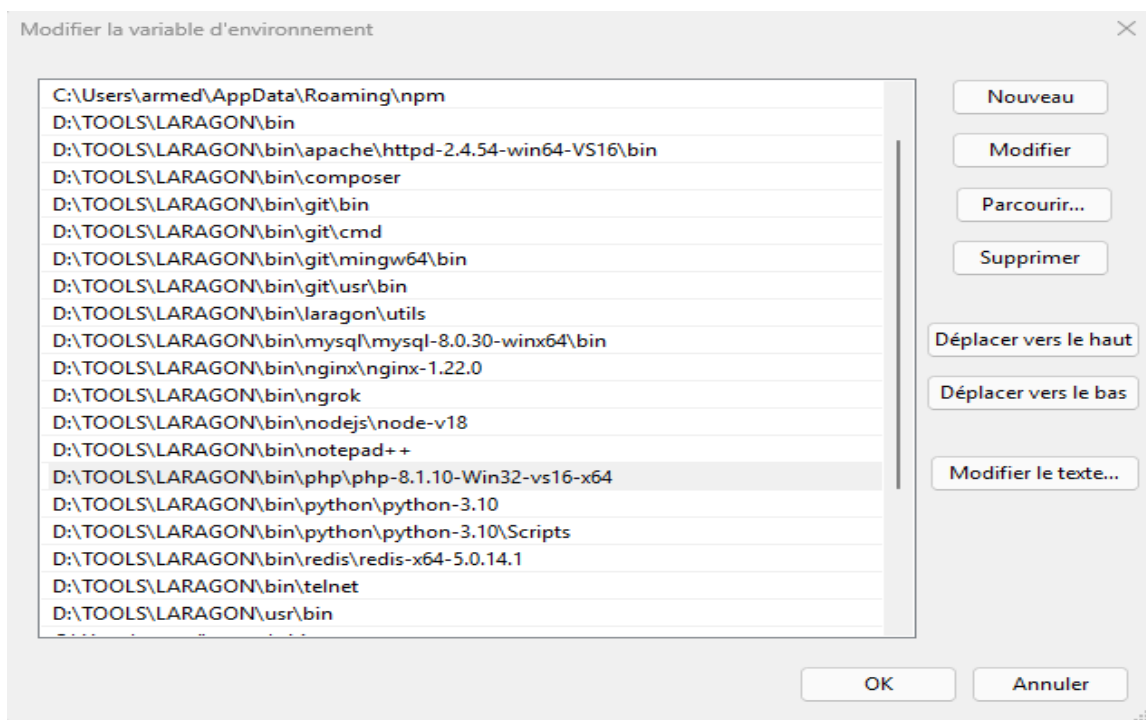


Figure 1.4 -Fenêtre « Modifier la variable d'environnement », avec la ligne du chemin PHP (php-8.1.10)

Après redémarrage du terminal, ``php -v`` affichait la version 8.1.10 : les commandes PHP étaient désormais accessibles, et ce depuis n'importe quel dossier, indépendamment de l'interface graphique de Laragon.

```
GCA@LGram MINGW64 /d/TOOLS/LARAGON/www/livre-or
$ php -v
PHP 8.1.10 (cli) (built: Aug 30 2022 18:05:49) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.1.10, Copyright (c) Zend Technologies

GCA@LGram MINGW64 /d/TOOLS/LARAGON/www/livre-or
$ git --version
git version 2.51.0.windows.1
```

Figure 1.5 -Terminal VS Code affichant la sortie de `php -v` et `git --version`.

DOSSIER PROFESSIONNEL ^(DP)

Une fois l'environnement prêt, je me suis placé dans mon dossier de travail (racine du serveur, répertoire « www » de Laragon), j'ai initialisé le suivi de version avec ``git init``, puis configuré la liaison avec mon dépôt distant préalablement créé sur GitHub en exécutant la commande appropriée:

```
git remote add origin https://github.com/geoffrey-carpentier/Livre-or.git
```

Afin de garantir la confidentialité des données, j'ai créé un fichier ``.gitignore`` avant d'effectuer mon premier commit. Cette étape est cruciale pour exclure le fichier ``.env`` du suivi de version, car il contient les informations de connexion à la base de données. En prenant cette mesure de sécurité, j'ai évité de pousser des informations sensibles sur GitHub.

Une fois ce circuit intégralement opérationnel, incluant le code en local, la sauvegarde et le partage via le dépôt distant, j'ai pu me consacrer pleinement au développement de mon Livre d'Or au sein d'un environnement sécurisé.

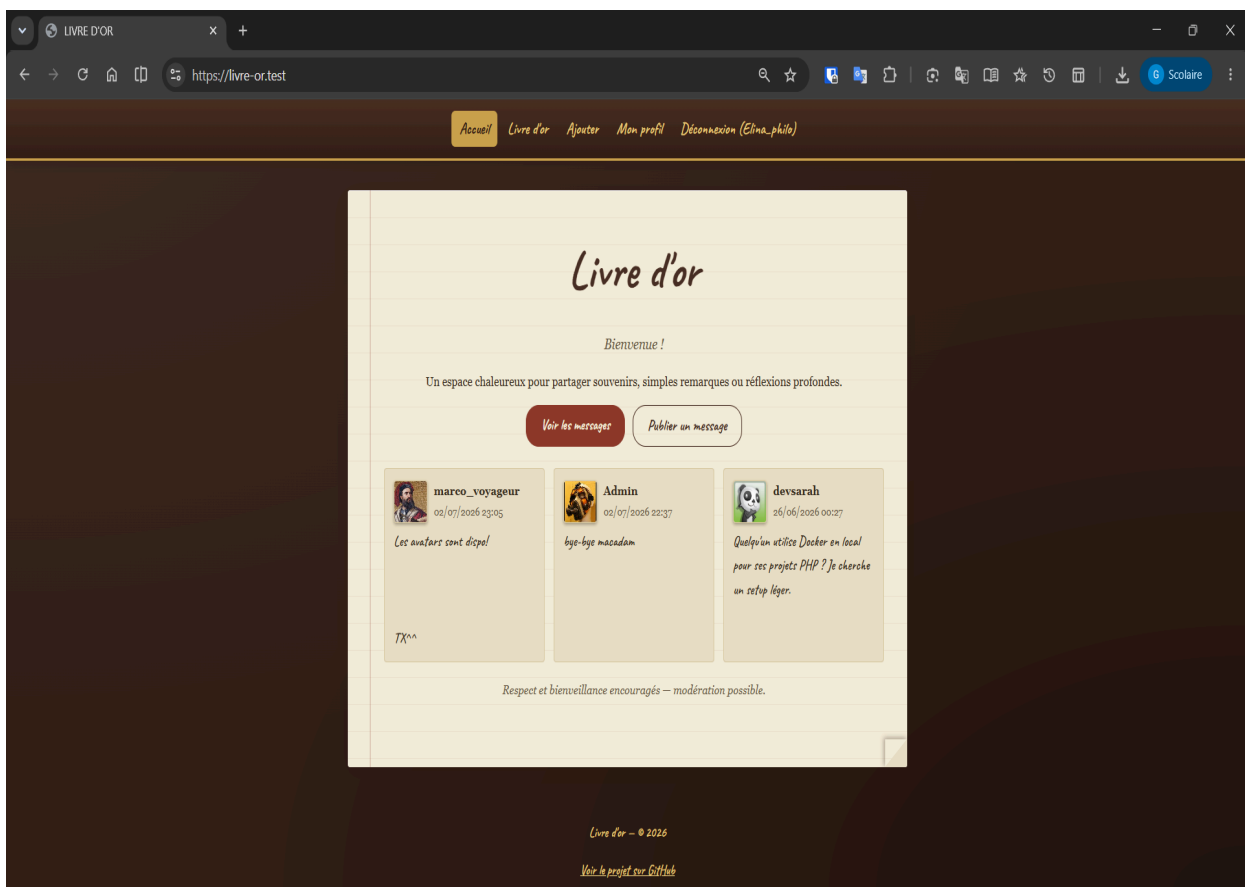


Figure 1.6 - Mon Livre d'or, affiché dans le navigateur Chrome

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

- **Matériels et logiciels** : poste sous Windows 11, Visual Studio Code, Git (avec Git Bash), pile WAMP via Laragon, MySQL et phpMyAdmin.
- **Technologies et configurations** : PHP 8.1.10 (ajout du chemin à la variable Path), HTTPS local (certificat TLS auto-signé Laragon), gestion de version avec Git et dépôt GitHub.

3. Avec qui avez-vous travaillé ?

J'ai réalisé l'intégralité des phases d'installation, de paramétrage et de résolution de problèmes liées à mon espace de travail. J'ai bénéficié pour cela du suivi pédagogique de mon formateur, Jean-Ely GENDRAU, qui s'est tenu à ma disposition pour m'orienter et m'épauler en cas de besoin.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme (Toulon)

Chantier, atelier, service ▶ Projet individuel « Livre d'or »

Période d'exercice ▶ Du : 03/11/2025 au : 07/11/2025

5. Informations complémentaires *(facultatif)*

La difficulté majeure rencontrée a été l'erreur lors de l'exécution de la commande `php -v`, due à un défaut de paramétrage de la variable d'environnement Path. Au-delà de la résolution technique, cette expérience m'a permis de mieux comprendre comment Windows gère la localisation des exécutables et pourquoi un programme peut ne pas être reconnu par le terminal. J'ai pris conscience que cette configuration locale rendait l'environnement difficilement transférable. Ce constat m'a incité à m'intéresser à Docker, afin de concevoir des environnements de travail plus facilement reproductibles.

Dépôt GitHub du projet : <https://github.com/geoffrey-carpentier/livre-or>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ➤

Maquetter des interfaces utilisateur web ou web mobile (CP2)
Réaliser des interfaces utilisateur statiques web ou web mobile (CP3)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pendant deux mois, j'ai participé au développement de la plateforme web du festival MarsAI, consacré aux courts-métrages générés par intelligence artificielle. J'étais chargé de la conception des interfaces puis de leur intégration, dans une équipe de six développeurs.

La difficulté tenait au nombre de profils à servir. Quatre publics devaient cohabiter sur le même site : les visiteurs qui viennent regarder les films, les réalisateurs qui déposent une candidature, les membres du jury qui notent les œuvres qu'on leur a attribuées, et les administrateurs qui pilotent l'ensemble. Chacun a besoin d'écrans différents, et certains ne doivent jamais voir ceux des autres.

J'ai donc commencé par cadrer les parcours avant de dessiner quoi que ce soit. Nous avons tenu un atelier collectif sur FigJam pour poser l'arborescence du site et suivre, profil par profil, le chemin d'un utilisateur d'un bout à l'autre de son usage. C'est ce travail qui a rendu lisibles les deux cas les plus compliqués : le formulaire de candidature des réalisateurs, découpé en quatre étapes, et le circuit de notation du jury.

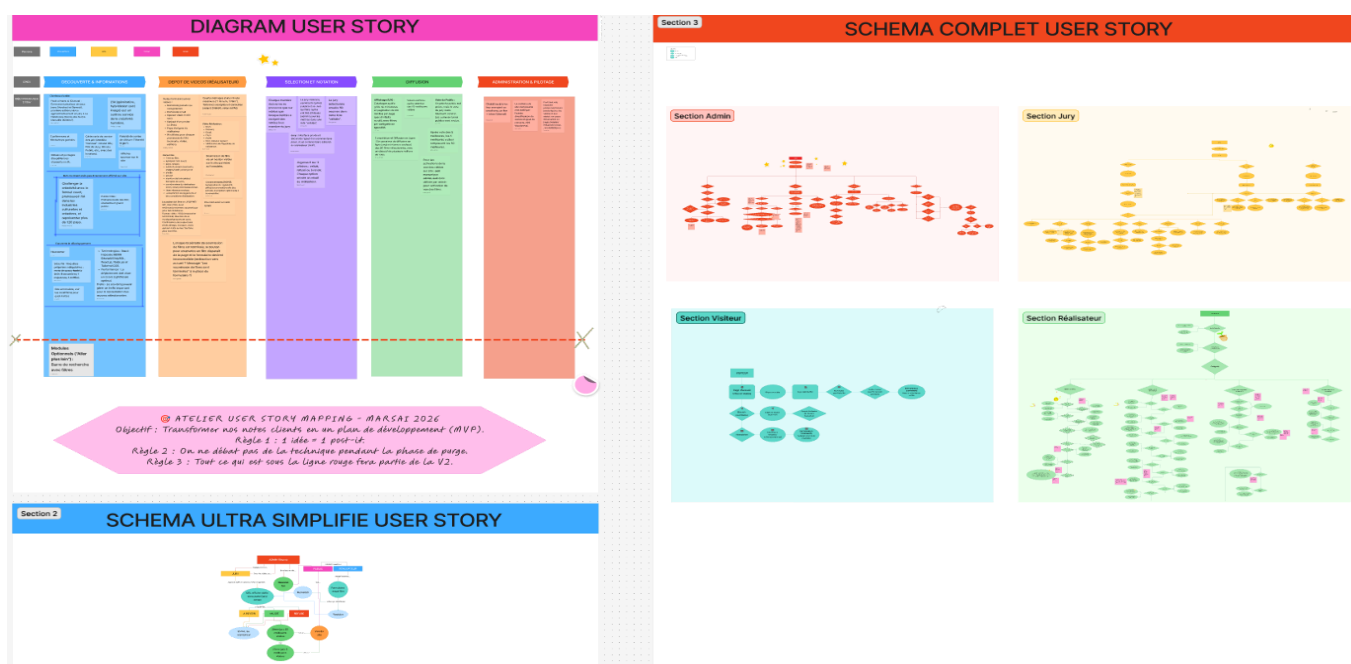


Figure 2.1 : Atelier de cadrage et parcours utilisateurs ("User Stories") sur FigJam.

Ces parcours ont ensuite été formulés sous forme de “user stories”, que nous avons classées par priorité selon la méthode MoSCoW. L'exercice oblige à distinguer ce qui est indispensable au lancement (Must) de ce qui est souhaitable, voire qui attendra (Should/Could/Would). Nous avons retenu quatre fonctions comme indispensables: la consultation du catalogue et le visionnage pour le visiteur, le dépôt de candidature pour le réalisateur, la notation des films attribués pour le jury, et la gestion des comptes et des affectations pour l'administrateur.

Design d'interface et prototypage adaptatif

J'ai ensuite entrepris la conception de maquettes sous Figma, en commençant par le format téléphone (approche “**Mobile-First**”). Dessiner d'abord pour un petit écran présente certaines contraintes inhérentes au format réduit et oblige à hiérarchiser : quand il n'y a pas la place pour tout montrer, il faut savoir prioriser, déterminer ce qui compte le plus. Une fois la fondation mobile consolidée, j'ai décliné et étendu ensuite vers les formats tablette et le bureau, en prenant 1440 × 1024 px comme résolution de référence. J'ai procédé en deux temps, des maquettes en fil de fer (**wireframes basse fidélité**) pour fixer l'emplacement des blocs, puis une maquette détaillée et cliquable haute fidélité, qui a servi à valider les écrans avec l'équipe avant d'écrire la première ligne de code, ouvrant ainsi la voie à la phase d'intégration technique.

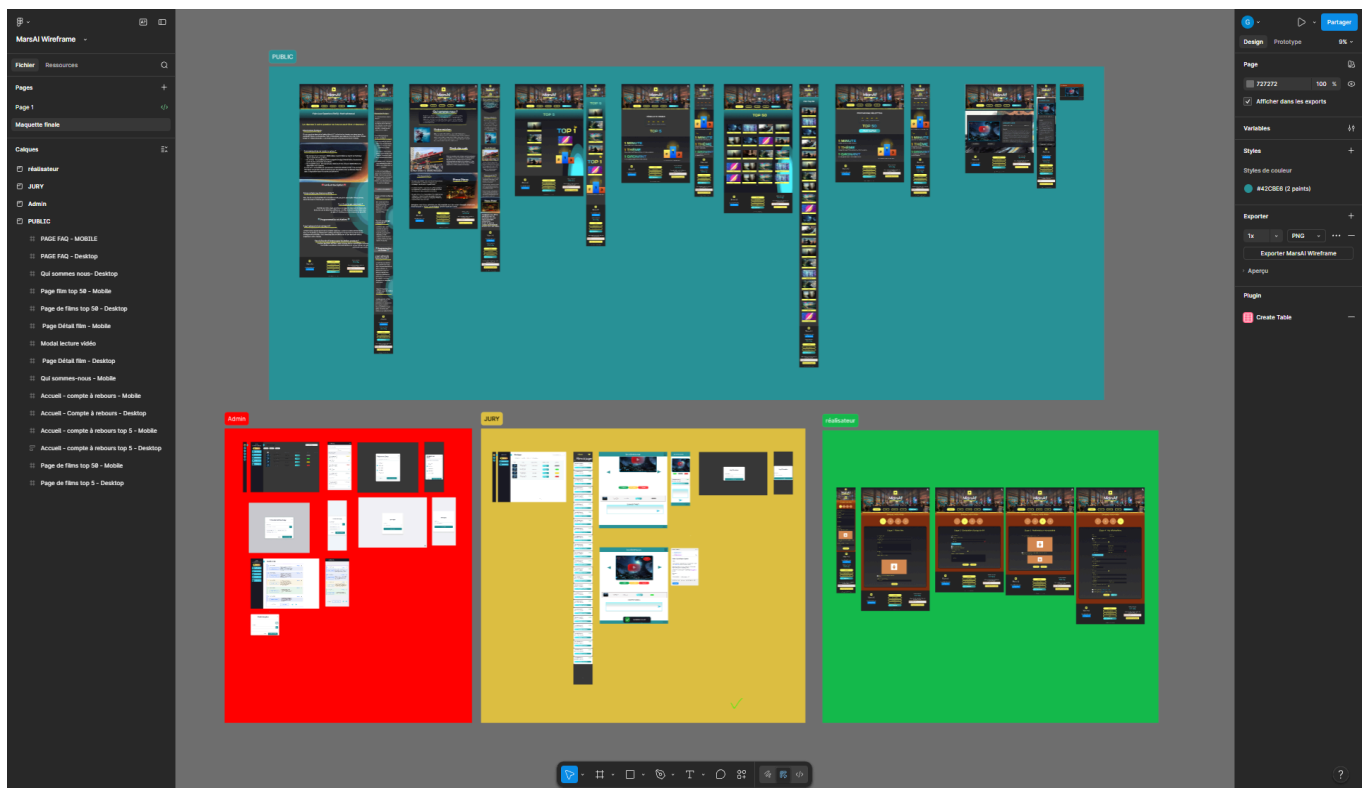


Figure 2.2 - Vue des maquettes Figma par profil (Public/Admin/Jury/Réalisateur) aux formats mobile et bureau.

Ce travail a permis de produire un système de composants : une palette pour le mode sombre, des règles de typographie, ainsi qu'une bibliothèque de boutons, de cartes et de champs réutilisables. Son intérêt n'est pas seulement visuel. En posant les composants avant l'intégration, on sait à l'avance ce qu'il faut coder, ce qui permet d'éviter par exemple que deux développeurs écrivent deux boutons différents pour le même usage.



Figure 2.3 - Système de composants Figma : palette, typographie et éléments réutilisables.

Intégration des interfaces utilisateur statiques (CP 3)

Le travail de maquettage achevé a laissé place à la phase d'intégration. Pour ce faire, j'ai suivi les maquettes composant par composant, en **React 19** avec **Vite** pour la construction et **Tailwind CSS 4** pour la mise en forme. Les classes utilitaires de Tailwind portent directement les points de rupture, ce qui évite de tenir une feuille de style séparée pour chaque largeur d'écran. Trois gabarits de page spécifiques (un public, un pour le jury et un pour l'administration) structurent l'application en garantissant une cohérence visuelle, quand **React Router** assure la navigation fluide, sans rechargement, propre aux Single Page Applications (SPA). Le site devant être bilingue pour répondre au caractère international de l'évènement, la bascule français-anglais est pilotée dynamiquement par la bibliothèque **i18next**.

À ce stade, deux points ont demandé une attention particulière :

- L'accessibilité d'abord : j'ai employé les balises structurantes de HTML5 plutôt que des div génériques, et ajouté les attributs ARIA là où la structure ne suffisait pas, en contrôlant le résultat avec Lighthouse et l'extension Axe.
- La sécurité ensuite : React échappe automatiquement ce qu'il affiche, ce qui protège des injections de scripts intersites (**XSS**), à condition de ne pas court-circuiter ce mécanisme. Je me suis donc interdit *dangerouslySetInnerHTML*, qui aurait rouvert la faille.

Pour finaliser cette étape d'intégration technique, je me suis assuré que le rendu du site restait bien fidèle aux maquettes de référence. J'ai ainsi testé l'affichage responsive sur plusieurs résolutions pour vérifier que l'interface s'adaptait correctement, depuis le mobile jusqu'au grand écran :

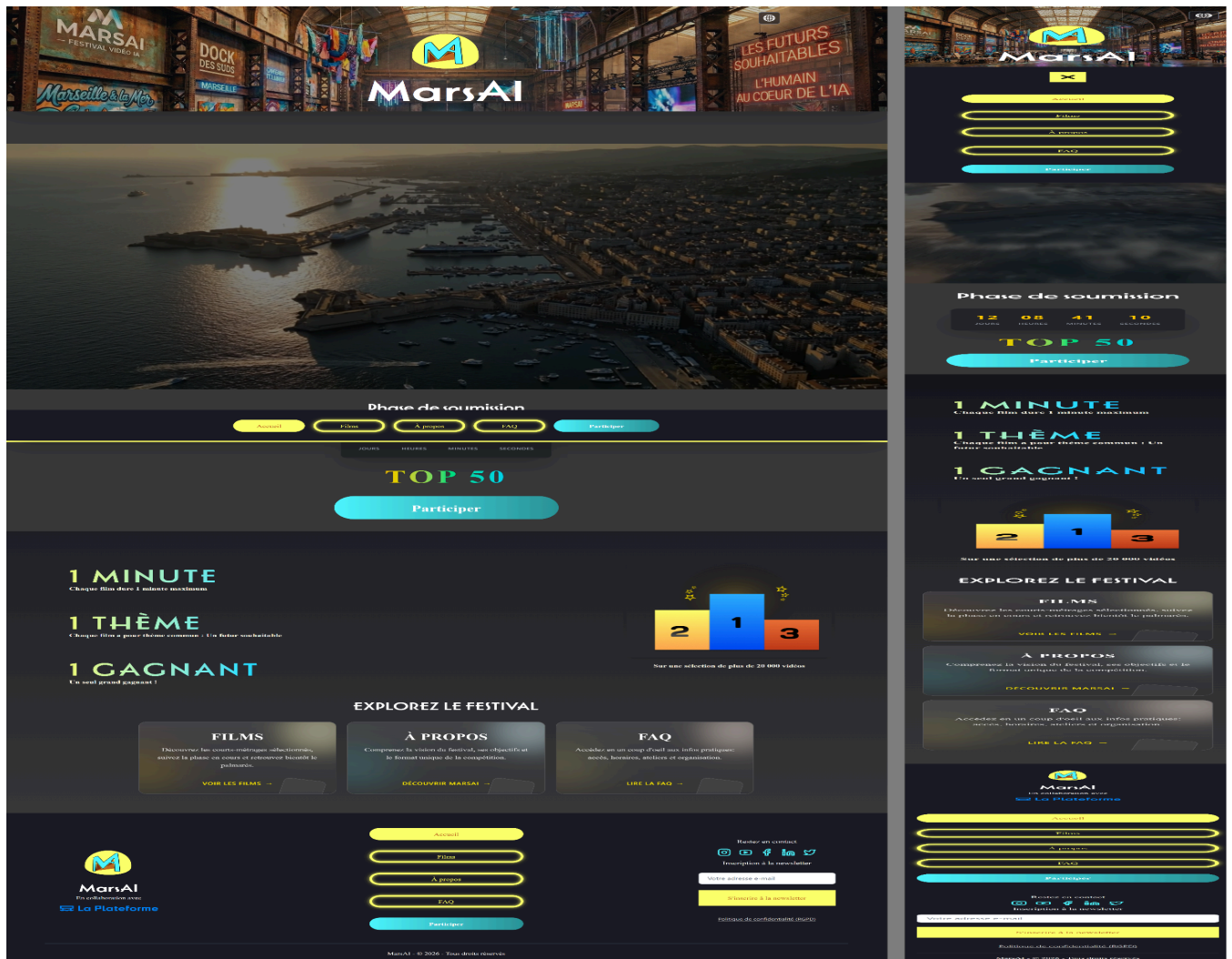


Figure 2.4 : Rendu de la page d'accueil, en bureau (gauche) et en mobile (droite).

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

- **Matériels et logiciels** : poste sous Windows 11, Visual Studio Code, FigJam (ateliers de cadrage), Figma (maquettage et Design System), navigateur avec outils d'audit (Lighthouse, extension Axe).
- **Stack technique (CP2/CP3)** : React 19 (Hooks, JSX) avec Vite 7 pour le build et le Hot Module Replacement ; Tailwind CSS 4 pour la mise en page responsive.
- **Routage et internationalisation** : React Router 7 pour la navigation en mode SPA ; i18next pour la prise en charge bilingue (français/anglais).
- **Qualité et accessibilité** : audits Lighthouse et extension Axe DevTools pour valider la sémantique HTML5/ARIA et les contrastes (WCAG AA).
- **Gestion de version et travail collaboratif** : Git en ligne de commande, GitHub pour la centralisation du code et la résolution des conflits de fusion.

3. Avec qui avez-vous travaillé ?

Intégré à une équipe polyvalente de six développeurs, j'ai collaboré sur l'ensemble du projet (design, front-end, back-end), me concentrant activement sur le design UI/UX et l'intégration front-end. Les choix techniques et de conception étaient toujours collectifs. Si cette organisation a multiplié les conflits de fusion Git, elle a grandement favorisé le partage de compétences et à ce titre s'est avérée particulièrement formatrice.

4. Contexte

Nom de l'entreprise, organisme ou association - La Plateforme (Toulon)
Chantier, atelier, service - Projet collaboratif MarsAI : Festival du film IA
Période d'exercice - Du 12/03/2026 au 06/05/2026

5. Informations complémentaires (facultatif)

L'audit final Lighthouse a révélé une baisse de performance sur la page d'accueil, causée par une vidéo en arrière-plan et des images brutes de plusieurs mégaoctets ralentissant le chargement (simulé sur mobile bridé). Pour y remédier, trois optimisations prioritaires ont été définies : compression vidéo, conversion des visuels en WebP adaptatif et application du chargement différé (*lazy-loading*).

Même si l'approche *mobile-first* a garanti une expérience utilisateur globalement fluide et cohérente, la réalisation de tests utilisateurs préalables sur la maquette Figma aurait aidé à détecter les frictions plus précocement et à optimiser les parcours de navigation.

Livrables et ressources du projet : [Maquettes Figma MarsAI](#) - [Schéma FigJam](#)

Dépôt GitHub : <https://github.com/geoffrey-carpentier/mars-ai-grp2>

Activité-type 1

Exemple n°3 ▶

Développer la partie front-end d'une application web ou web mobile sécurisée

Développer la partie dynamique des interfaces utilisateur web ou web mobile (CP4)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

La création d'un jeu de Memory a été pour moi l'occasion de développer une interface front-end fluide et réactive aux actions du joueur garantissant une application stricte des règles du jeu établies.

Le principal défi de ce projet consistait à synchroniser en temps réel l'état logique du jeu et l'affichage des cartes pour éviter tout bug lié à des clics trop rapides. Avant de concevoir l'aspect visuel, j'ai donc structuré l'état global de l'application en définissant les variables clés de sa gestion : le verrouillage des interactions, le suivi des cartes sélectionnées et le compteur de paires trouvées.

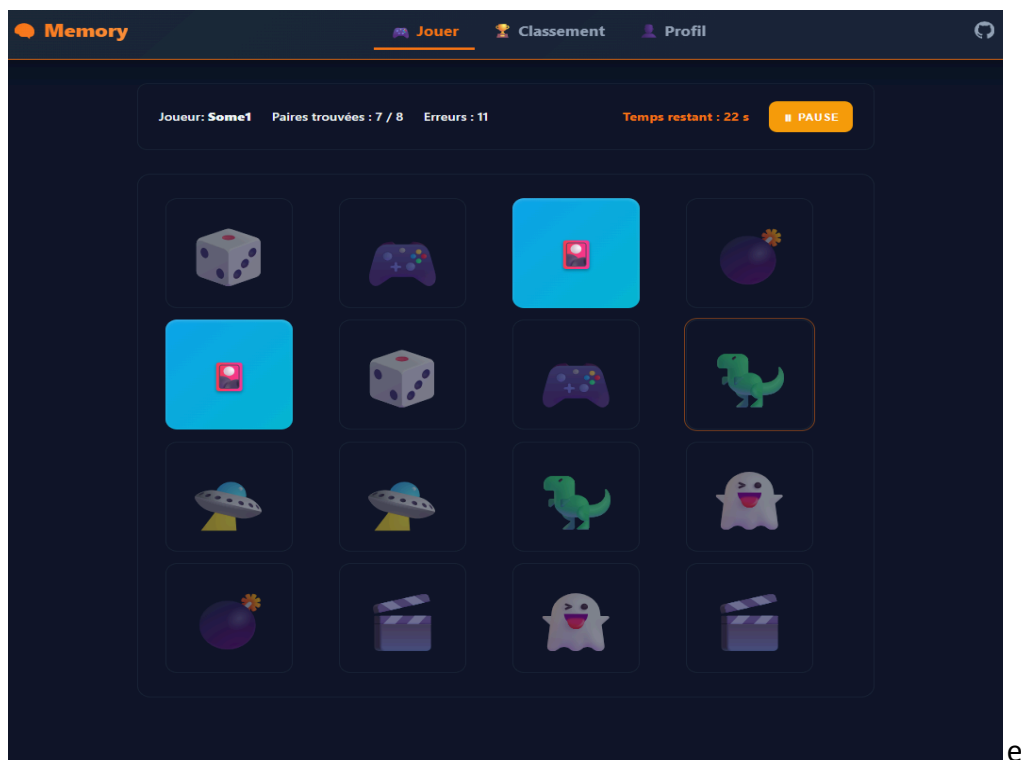


Figure 3.1 - Le plateau de jeu en cours : chronomètre, compteurs (paires / erreurs) et cartes retournées.

Une fois tous les éléments représentant les cartes récupérés dans le DOM, des écouteurs d'événements au clic y sont attachés. La gestion globale du retournement est ainsi centralisée par l'intermédiaire d'une fonction unique : **handleFlip**.

Le fonctionnement du système d'appariement s'articule autour des aspects suivants :

- **Identification et comparaison** : L'attribut HTML personnalisé *data-pair* sert à valider la correspondance entre deux cartes.
- **Rendu et animations visuelles** : L'animation de retournement 3D est prise en charge par des règles CSS3 (notamment *perspective*, *transform: rotateY* et *backface-visibility: hidden*).
- **Validation des paires** : La découverte d'une paire identique entraîne l'ajout de la classe *is-matched* et l'incrémentation du nombre de paires validées.
- **Temporisation et contrôle des erreurs** : Si les deux cartes diffèrent, la variable *lockBoard* interdit toute nouvelle sélection. Une pause de 900 ms créée par *setTimeout* laisse à l'utilisateur le temps de mémoriser les cartes avant qu'elles ne se retournent et que le plateau ne soit déverrouillé. Ce verrou empêche de retourner une troisième carte prématurément, même s'il ne protège pas contre les modifications directes exécutées côté client.

```
function handleFlip(card) {
  if (
    lockBoard ||
    card.classList.contains("is-flipped") ||
    card.classList.contains("is-matched")
  ) {
    return;
  }

  card.classList.add("is-flipped");
  sound?.flip();

  if (!firstCard) {
    firstCard = card;
    return;
  }

  const isMatch = firstCard.dataset.pair === card.dataset.pair;

  if (isMatch) {
    firstCard.classList.add("is-matched");
    card.classList.add("is-matched");
    pairsFound++;

    if (statPairs) {
      statPairs.textContent = `Paires trouvées : ${pairsFound} / ${totalPairs}`;
    }

    sound?.match();
    showToast(randomFrom(messages.success), "success");
    firstCard = null;

    if (pairsFound === totalPairs) {
      finishForm.querySelector('input[name="result_type"]').value = "win";
      sound?.win();
      setTimeout(endGame, 500);
    }
  } else {
    errors++;
    if (statErrors) {
      statErrors.textContent = `Erreurs : ${errors}`;
    }

    sound?.error();
    showToast(randomFrom(messages.fail), "fail");
    lockBoard = true;

    setTimeout(() => {
      firstCard.classList.remove("is-flipped");
      card.classList.remove("is-flipped");
      firstCard = null;
      lockBoard = false;
    }, 900);
  }
}
```

Figure 3.2 - Extrait de *memory.js* : comparaison des cartes et blocage temporaire des retournements.

Pour préserver l'équité du jeu et contrer les tentatives d'anti-jeu, comme la mise en pause frauduleuse du chrono, l'API *Page Visibility* a été mise en œuvre. L'écoute de l'événement *visibilitychange* détecte le passage de l'onglet en arrière-plan, ce qui stoppe immédiatement le compte à rebours tout en économisant les ressources du système.

La partie prend fin lorsqu'un joueur découvre toutes les paires (victoire), ou quand le chronomètre tombe à zéro. En cas de succès, l'annonce de la victoire s'affiche dès la confirmation de la dernière paire.

En appuyant directement ce contrôle sur le total de paires déjà validées, le système s'affranchit d'une vérification complète de l'ensemble du plateau.

Une fois la partie finie, l'écran de bilan récapitule les différentes performances de la session (nombre de paires validées, fautes commises et temps restant) avant d'expédier ces résultats par une requête POST vers le serveur PHP.



Figure 3.3 : Bilan d'une partie terminée, après le calcul du score côté serveur.

Le score final est calculé côté serveur, à partir des métriques transmises. Il tient compte de la difficulté de la partie (nombre de paires à trouver), des erreurs commises, et de la rapidité du joueur (temps restant en fin de partie). Il est enregistré en base et intégré au classement des 10 meilleurs joueurs, consultable à tout moment sur la page dédiée.

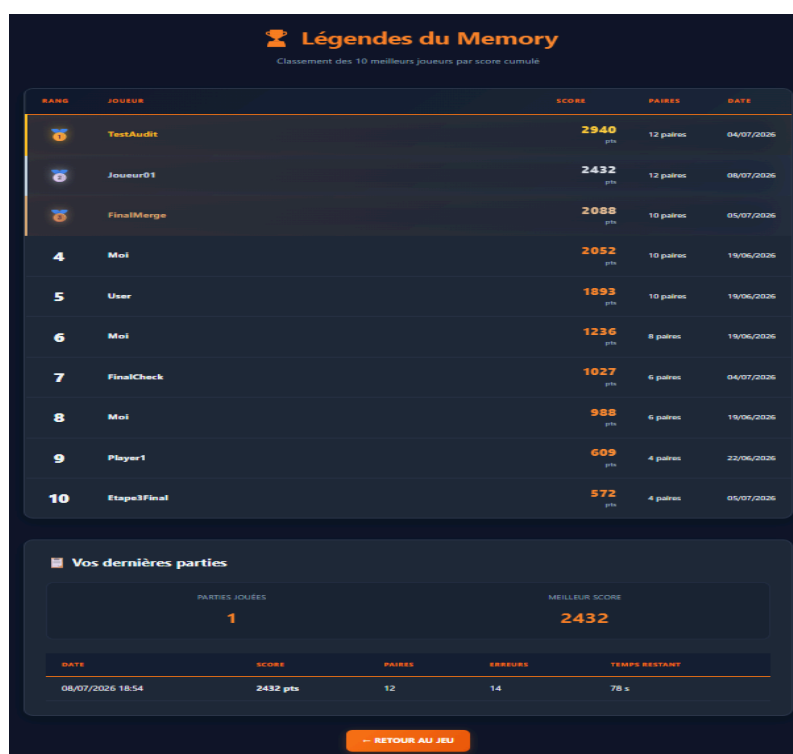


Figure 3.4 - Classement des meilleures parties et historique personnel du joueur.

2. Précisez les moyens utilisés :

- **Matériels et IDE** : Ordinateur sous Windows 11, Visual Studio Code.
- **Outils de test et environnement local** : Navigateur Web équipé des Chrome DevTools (analyse de la console et inspection des écouteurs d'événements), pile serveur locale Laragon.
- **Technologies Front-End** : JavaScript ES6 (manipulation dynamique du DOM, programmation asynchrone via setTimeout / setInterval, API Page Visibility), HTML5 (data-attributes), CSS3 (animations et transformations 3D).
- **Technologies Back-End** (Support de persistance) : Langage PHP (architecture MVC pour la réception des métriques), SGBD MySQL (stockage des scores).

3. Avec qui avez-vous travaillé ?

J'ai conçu et développé l'intégralité de la logique algorithmique, de l'interface graphique et de la liaison serveur de manière totalement autonome, en tant que développeur unique sur ce projet.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme (Toulon)

Chantier, atelier, service ▶ projet individuel « Memory »

Période d'exercice ▶ Du : 19/11/2025 au : 28/11/2025

5. Informations complémentaires *(facultatif)*

Avec le recul, ce projet m'a surtout permis de mesurer l'importance de la gestion d'état dans une interface dynamique. Même sur une application relativement simple, quelques variables bien choisies suffisent à garantir un comportement fiable. Si je devais faire évoluer ce projet aujourd'hui, j'en profiterais pour structurer davantage le code sous forme de modules ES6 afin d'améliorer sa maintenabilité.

Une autre limite est que toute la logique est concentrée dans une seule fonction, mélangeant ainsi la manipulation du DOM et les règles métier. Pour améliorer le code, il serait préférable de séparer la logique pure dans des modules distincts, ce qui permettrait aussi de réaliser des tests unitaires (via Jest). Enfin, pour des projets plus vastes, l'utilisation d'une bibliothèque réactive comme React simplifierait la gestion des états complexes.

Dépôt GitHub du projet : <https://github.com/geoffrey-carpentier/memory>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1

Mettre en place une base de données relationnelle (CP5)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet collaboratif « MarsAI » (la plateforme de catalogage et de jury pour des courts-métrages générés ou assistés par IA), j'ai travaillé à la conception et l'implémentation de la couche de persistance des données.

L'enjeu principal consistait à modéliser un schéma relationnel capable d'articuler des entités métiers interconnectées : les films, les profils des réalisateurs, les évaluations du jury, les technologies d'intelligence artificielle employées, les médias d'illustration et la communication.

Pour structurer ces données, j'ai dans un premier temps construit un **Modèle Conceptuel de Données** (MCD) visant à représenter les principales informations manipulées par l'application et les liens qui les unissaient. Cette phase d'analyse conceptuelle m'a permis de valider la couverture des besoins fonctionnels de l'application avant d'aborder les contraintes techniques. L'étude du cahier des charges a conduit à l'identification de **11 entités principales** liées par des associations sémantiques précises:

- **movies** : l'entité centrale regroupant les métadonnées de l'œuvre (titres original et anglais, fichier vidéo, prompt IA, synopsis, durée, classement Top 5).
- **users** : les comptes des membres du jury et des administrateurs de la plateforme, authentifiés par un jeton d'accès unique.
- **director_profile** : la fiche descriptive du réalisateur, connectée à un film par une relation un-à-un (1:1).
- **comments** : les évaluations et retours d'expérience du jury, configurables en mode public ou privé via un champ dédié (isprivate).
- **ai_list**: le référentiel des solutions d'IA disponibles sur le marché.
- **status** : le référentiel d'état fixant le niveau de modération d'une œuvre (en attente, validé, rejeté).
- Les entités de support pour les médias et la communication : **sound_data**, **screenshots**, **socials**, **email** et **newsletter**.

Ce modèle conceptuel a été soumis à l'équipe et validé collectivement avant de passer à l'étape de modélisation suivante.

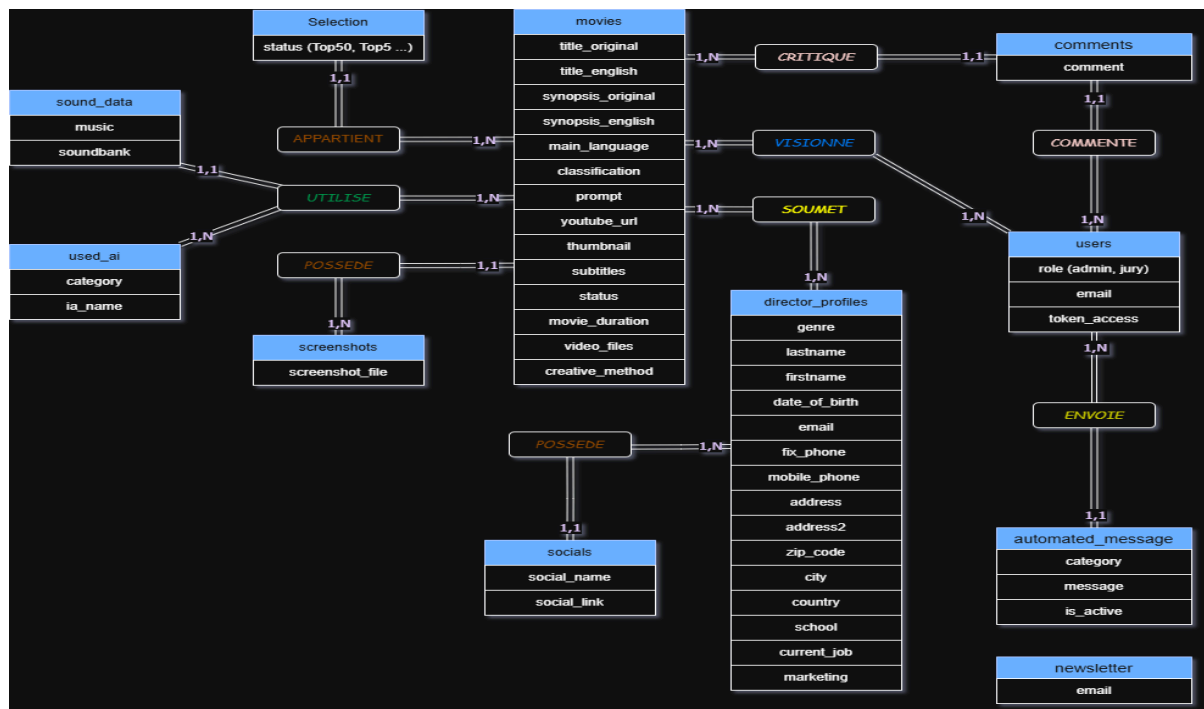


Figure 4.1 — Modèle Conceptuel de Données MarsAI, [Draw.io](https://draw.io).

À partir de cette structure validée, j'ai élaboré le Modèle Logique de Données (MLD). Cette transition a matérialisé mes entités en tables physiques et résolu les associations complexes. Les relations de type "plusieurs-à-plusieurs" (N:N) ont ainsi été transformées en **2 tables de jonction autonomes** : **used_ai** (pour associer un film aux différents outils d'IA utilisés) et **users_movies** (pour gérer les affectations des films aux jurés), ce qui a permis d'optimiser le stockage et de garantir de bonnes performances lors des futures requêtes.

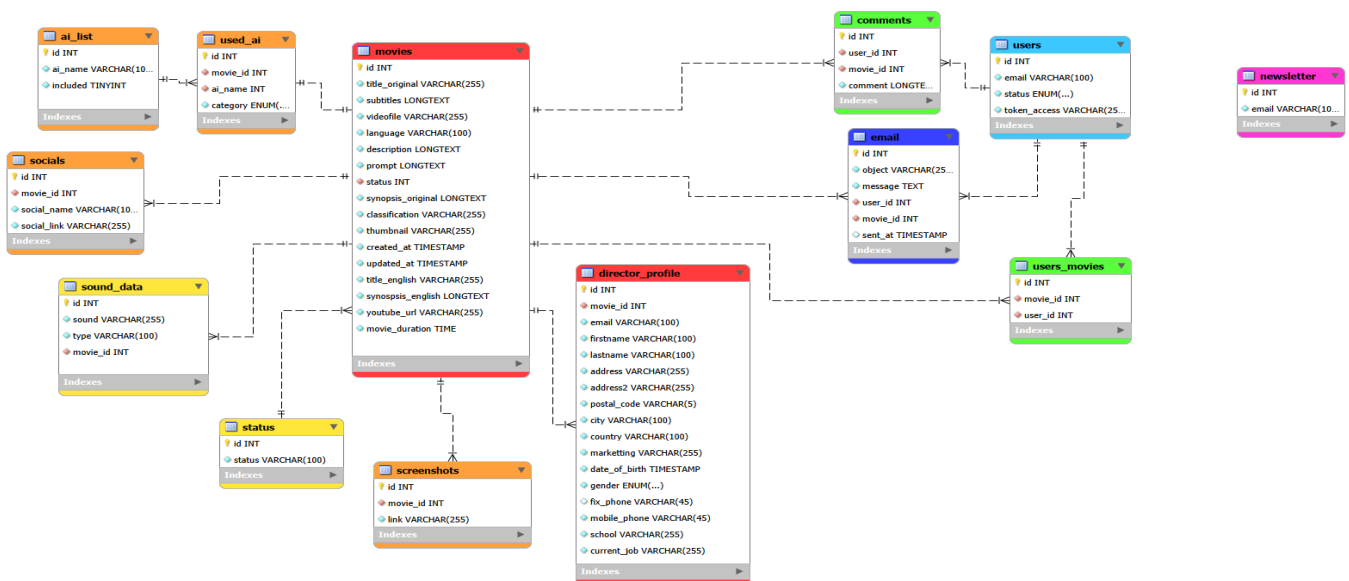


Figure 4.2 — Modèle Logique de Données MarsAI (MySQL Workbench)

Pour la conception du MLD, j'ai traduit les relations de type 1:N en clés étrangères standards (comme `comments.movie_id` pointant vers `movies.id`). L'ensemble de ce schéma finalisé à **13 tables** respecte rigoureusement la *Troisième Forme Normale (3NF)* : chaque attribut non-clé dépend uniquement et directement de la clé primaire de sa table, éliminant ainsi tout risque de dépendance transitive et de redondance de stockage.

La traduction de ce modèle logique en Langage de Définition de Données (DDL) a abouti à la création du script d'initialisation `schema.sql` sous **MySQL 8.0.30**. Pour l'implémentation physique, j'ai retenu plusieurs choix structurants : l'utilisation exclusive du moteur de stockage **InnoDB**, indispensable pour assurer la gestion native des clés étrangères et la sécurité des transactions **ACID** ; le jeu de caractères **utf8mb4** (collation `utf8mb4_0900_ai_ci`) pour garantir une prise en charge native et complète de l'Unicode (titres en langues étrangères et caractères spéciaux) ; et enfin l'ajout de contraintes FOREIGN KEY associées à des clauses ON DELETE CASCADE sur les données dépendantes. Ainsi, la suppression d'un film automatise le nettoyage de ses commentaires et captures d'écran associés, maintenant l'intégrité référentielle du SGBD sans surcharge applicative.

```
-- Entité centrale : les films
CREATE TABLE `movies` (
  `id` int NOT NULL AUTO_INCREMENT,
  `title_original` varchar(255) NOT NULL,
  `status` int NOT NULL DEFAULT '1',
  `prompt` longtext NOT NULL,
  `movie_duration` time NOT NULL,
  `top5_rank` tinyint DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_movies_status`
    FOREIGN KEY (`status`) REFERENCES `status` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Table de jonction N:N : un film peut utiliser plusieurs outils d'IA,
-- avec la catégorie d'usage (scénario, image/vidéo, post-production)
CREATE TABLE `used_ai` (
  `id` int NOT NULL AUTO_INCREMENT,
  `movie_id` int NOT NULL,
  `ai_name` int NOT NULL,
  `category` enum('script','movie','postprod') NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_usedai_movie`
    FOREIGN KEY (`movie_id`) REFERENCES `movies` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_usedai_ai`
    FOREIGN KEY (`ai_name`) REFERENCES `ai_list` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Figure 4.3 — Reconstruction simplifiée des tables et contraintes du script `schema.sql`, à des fins de lisibilité (les clés primaires et étrangères sont en réalité ajoutées via des instructions ALTER TABLE distinctes).

2. Précisez les moyens utilisés :

- **Matériels et logiciels** : poste sous Windows 11, serveur local Laragon (MySQL 8.0.30), phpMyAdmin pour l'export du schéma, *Draw.io* pour la modélisation du MCD et *MySQL Workbench* pour le MLD.
- **Technologies et configurations** : méthode Merise (MCD, MLD, DDL), moteur InnoDB (clés étrangères, transactions), jeu de caractères utf8mb4.

3. Avec qui avez-vous travaillé ?

J'ai travaillé sur l'architecture de cette base de données en collaboration au sein d'une équipe de six développeurs. Pour garantir la cohérence des développements backend et frontend de mes collègues, chaque étape clé de la modélisation (validation du MCD et des relations métiers...) a fait l'objet d'un arbitrage et d'une validation collective avant d'exécuter les scripts de création en base.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme (Toulon)

Chantier, atelier, service ▶ Projet collaboratif « MarsAI : Festival du film IA »

Période d'exercice ▶ Du : 01/03/2026 au : 04/06/2026

5. Informations complémentaires (facultatif)

La principale complexité de modélisation rencontrée ici résidait dans la relation plusieurs-à-plusieurs (N:N) entre les films et les outils d'IA : un film pouvant mobiliser plusieurs technologies (Runway, Midjourney, Sora) et un outil pouvant servir sur plusieurs œuvres. La table de jonction `used_ai` a permis de résoudre cette relation tout en y intégrant une métadonnée contextuelle propre à chaque usage (le type d'intervention : scénario, vidéo ou post-production), sans dénormaliser la structure globale. Par ailleurs, pour la gestion des commentaires du jury, l'arbitrage s'est fait en faveur d'un simple champ booléen `isprivate` plutôt que de scinder les données en deux tables distinctes, simplifiant ainsi les requêtes côté serveur (une simple clause WHERE suffit).

Sur le plan de la sécurité, nous avons choisi architecturalement de ne stocker aucun mot de passe utilisateur en base pour le profil du jury. L'accès à leur espace privé repose sur un mécanisme de jetons d'accès aléatoires (`token_access` au format UUID) générés côté serveur. Ce choix technique fait également office de verrou de sécurité : la régénération du token en base permet d'invalidier et de révoquer instantanément tous les accès d'un juré en cas de compromission.

Dépôt GitHub : <https://github.com/Sofiane224434/mars-ai-grp2>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶

Développer des composants métier côté serveur (CP7)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai pris en charge le développement de la logique serveur d'une application de réservation de salles, avec l'objectif de mettre en place une architecture robuste, évolutive et facilement maintenable. Pour ce faire, j'ai structuré un routage modulaire à l'aide d'Express (express.Router()), organisé selon un modèle multicouche (Routes → Contrôleurs → Modèles). Cette approche m'a permis d'isoler strictement chaque composant applicatif, garantissant ainsi la lisibilité du code, la traçabilité des requêtes et la cohérence des réponses serveur.

```
backend/
├── config/
│   └── db.js                # Pool de connexions MySQL2 + query() + testConnection()
├── controllers/
│   ├── auth.controller.js   # register · login · getProfile (émission JWT)
│   └── reservation.controller.js # CRUD réservations + application des règles métier
├── middlewares/
│   └── auth.middleware.js    # Vérification du token JWT (Bearer)
├── models/
│   ├── user.model.js        # Requêtes SQL utilisateurs + hash bcrypt
│   └── reservation.model.js  # Requêtes SQL réservations (create/find/update/delete)
├── routes/
│   ├── auth.routes.js       # /api/auth → register, login, me (+ rate limiting)
│   └── reservations.routes.js # /api/reservations → CRUD, planning semaine, "mes résas"
├── validators/
│   └── reservation.validator.js # Règles métier : jours ouvrés, horaires 8h-19h, durée min. 1h
├── app.js                   # Construction de l'app Express (middlewares, routes, 404)
├── server.js                # Point d'entrée : connexion DB + démarrage du serveur
├── schema.sql               # Schéma de la base de données MySQL
├── package.json             # Dépendances & scripts npm
└── .env.example             # Modèle de variables d'environnement
```

Figure 6.1: Structure modulaire multicouche de l'API Express (Routes / Contrôleurs / Modèles).

Afin de sécuriser l'accès aux fonctionnalités métier, j'ai implémenté un middleware d'authentification systématique sur l'ensemble des routes exposées. Ce composant intercepte le jeton JWT (JSON Web Token) transmis dans l'en-tête Authorization: Bearer, en vérifie l'intégrité et la validité via la méthode `jwt.verify()`, puis charge le profil de l'utilisateur concerné dans le contexte de la requête. En cas de jeton manquant, invalide ou expiré, le middleware lève immédiatement une exception et renvoie une réponse d'erreur HTTP 401 standardisée.

```
const authMiddleware = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Token manquant' });
    }
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({ error: 'Utilisateur non trouvé' });
    }
    req.user = user;
    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expiré' });
    }
    return res.status(401).json({ error: 'Token invalide' });
  }
}
```

Figure 6.2 : Middleware d'authentification JWT (`auth.middleware.js`),
vérification du jeton et gestion des erreurs 401. .

Pour la gestion des réservations, j'ai fait le choix architectural de centraliser les contraintes d'affaires dans un composant de validation dédié (`reservation.validator.js`), évitant ainsi de surcharger mes contrôleurs. J'ai programmé ce validateur pour intercepter et filtrer chaque demande selon les règles métiers strictes définies dans le cahier des charges : vérification que le créneau visé n'est pas déjà passé, restriction aux seuls jours ouvrés (du lundi au vendredi), validation des horaires sur des heures pleines (minutes à 00) impérativement comprises dans la plage 8h–19h, et limitation de la réservation à une durée minimale d'une heure au sein d'une seule et même journée.

```
if (!isNotInThePast(start)) { // # 2bis. Pas de créneau passé
  errors.push("Impossible de réserver un créneau déjà passé");
}
if (!isWeekday(start) || !isWeekday(end)) { // # 3. Jours ouvrés uniquement
  errors.push("Réservations autorisées du lundi au vendredi uniquement");
}
if (!isWithinWorkingHours(start, false)) { // # 4. Horaires de travail 8h-19h
  errors.push(
    "L'heure de début doit être entre 8h et 19h (heure pleine)",
  );
}
if (!isWithinWorkingHours(end, true)) { // # 4. Horaires de travail 8h-19h
  errors.push("L'heure de fin doit être entre 8h et 19h (heure pleine)");
}
if (!isMinimumDuration(start, end)) { // # 5. Durée minimum 1h
  errors.push("La durée minimum est de 1 heure");
}
if (start.toDateString() !== end.toDateString()) { // # 6. Même journée
  errors.push("La réservation doit se faire sur une seule journée");
}
```

Figure 6.3. : Extrait de la logique de validation des contraintes métiers dans reservation.validator.js

Le cœur fonctionnel de l'application réside dans la détection préventive des conflits de réservation, garantissant qu'aucune salle ne puisse être allouée simultanément sur des créneaux qui se chevauchent. Avant toute persistance en base de données, la couche modèle exécute une vérification ciblant la salle demandée ; un créneau est considéré comme disponible si aucune réservation existante ne recouvre la plage horaire soumise. Lors d'une modification, j'ai configuré la fonction pour exclure la réservation en cours d'édition (excludeId), évitant ainsi un faux positif de conflit avec ses propres données.

```
async isSlotAvailable(startTime, endTime, roomId = 1, excludeId = null) {
  let sql = `
    SELECT COUNT(*) as count
    FROM reservations
    WHERE room_id = ?
      AND start_time < ?
      AND end_time > ?
  `;
  const params = [roomId, endTime, startTime];

  if (excludeId) {
    sql += " AND id != ?";
    params.push(excludeId);
  }

  const result = await query(sql, params);
  return result[0].count === 0;
},
```

Figure 4 : Algorithme de détection des chevauchements de créneaux dans le modèle.(reservation.model.js)

Lorsqu'un conflit temporel est détecté par cette méthode, le contrôleur interrompt immédiatement le cycle de vie de la requête et retourne au client un code HTTP 409 (*Conflict*), accompagné d'un message explicite.



Figure 6.5 : Rejet d'une réservation conflictuelle (Code HTTP 409) sous Postman.

Parallèlement, j'ai veillé à structurer plusieurs niveaux de protection pour sécuriser les composants métier (CP 7). Côté persistance, j'ai systématiquement employé des requêtes SQL préparées afin de neutraliser tout risque d'injection SQL. En amont, les données entrantes subissent une validation rigoureuse (vérification des champs obligatoires, limitation de la taille des chaînes à 255 caractères et intégrité des formats de date). Enfin, j'ai implémenté une règle d'autorisation stricte venant compléter l'authentification : seul le créateur initial d'une réservation dispose des droits nécessaires pour la modifier ou la révoquer. Toute tentative d'altération sur la réservation d'un tiers est immédiatement bloquée par le serveur et sanctionnée par le renvoi d'un code HTTP 403 (*Forbidden*).

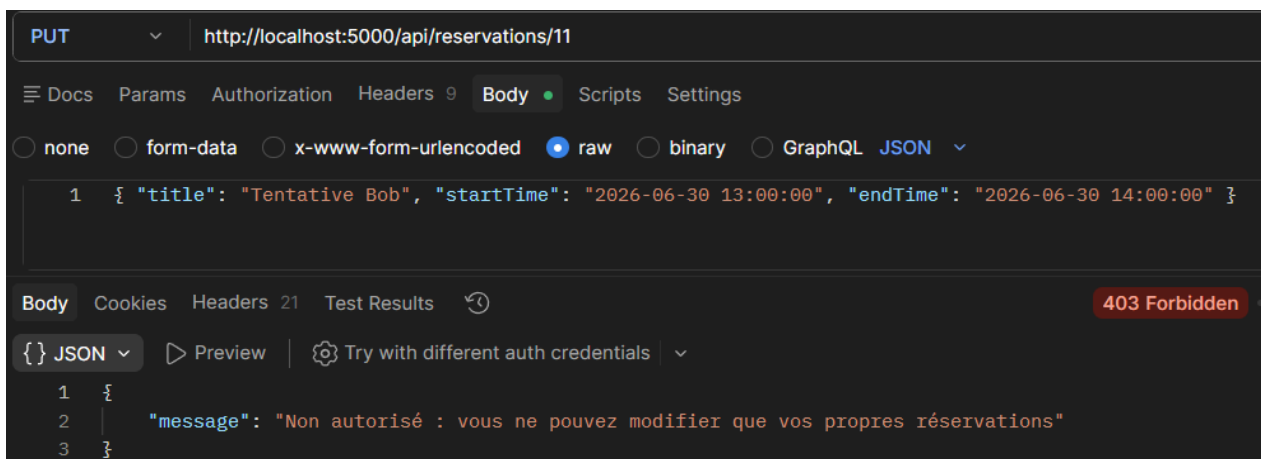


Figure 6.6 : Interception d'une modification non autorisée (Code HTTP 403) sous Postman

2. Précisez les moyens utilisés :

- **Matériels et logiciels** : poste sous Windows 11, Visual Studio Code, environnement Node.js avec Express, client de test d'API (Postman), MySQL (Laragon).
- **Technologies et configurations** : JavaScript ES6+ (objet natif Date) ; Express 5 (routage modulaire) ; pilote mysql2 avec requêtes préparées ; jsonwebtoken (JWT signé avec expiration) ; bcrypt (hachage des mots de passe, coût 10) ; architecture multicouche routes → contrôleurs → modèles.

3. Avec qui avez-vous travaillé ?

J'ai conçu et développé cette API métier de manière individuelle, de la définition des règles de gestion jusqu'aux algorithmes de contrôle, en passant par la sécurisation des accès.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme (Toulon)

Chantier, atelier, service ▶ API métier "Réservation-salles"

Période d'exercice ▶ Du : 09/02/26 au : 19/02/26

5. Informations complémentaires *(facultatif)*

Ce qui me semblait au départ un simple détail, la détection des chevauchements de créneaux, s'est révélé plus délicat que prévu. La difficulté portait sur les réservations contiguës : un créneau qui se termine à 10h ne doit pas être vu comme en conflit avec un autre qui commence à 10h. J'ai résolu ce cas avec des inégalités strictes : `start_time < finDemandée` et `end_time > débutDemandé`.

Dans une première version, j'avais utilisé `<=` à la place de `<`, ce qui rejetait à tort des réservations pourtant valides. Une fois corrigé, l'algorithme bloque bien les vrais chevauchements sans jamais gêner des créneaux qui se touchent simplement.

Côté sécurité, l'API combine authentification (JWT), autorisation (contrôle de propriété), requêtes préparées et validation systématique des entrées. La prochaine amélioration consistera à écrire une suite de tests unitaires (Jest) sur les cas limites du validateur (créneau passé, hors plage, à cheval sur deux jours), pour fiabiliser et documenter durablement ces composants métier.

Dépôt GitHub : <https://github.com/geoffrey-carpentier/Reservation-salles>

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°3 ▶

Développer des composants d'accès aux données SQL et NoSQL (CP6)
Documenter le déploiement d'une application dynamique web ou web mobile (CP8)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mon travail sur le projet « Médiathèque », une application web de gestion et de prêts de médias développée en PHP orienté objet avec une architecture MVC, j'ai articulé mon intervention autour de deux axes majeurs : la conception de composants d'accès aux données et le déploiement de l'application en environnement de production.

Pour que l'application puisse échanger avec notre base MySQL, j'ai utilisé l'extension native **PDO** de PHP. Afin d'éviter d'ouvrir une connexion à chaque action de l'utilisateur, notre accès a été configuré sous forme de **Singleton**. Cela garantit qu'une seule et unique connexion reste ouverte tout au long du chargement de la page.

```
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);  
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
```

L'enregistrement d'un emprunt est une opération à double effet combinant une insertion dans emprunts et un décrémentation du stock rendue atomique par une transaction ACID encapsulée dans un try/catch :

```
function create_emprunt($user_id, $media_id) {  
    db_execute("START TRANSACTION");  
    try {  
        // Insertion de l'emprunt  
        $date_retour_prevue = date('Y-m-d', strtotime('+14 days')); // 14 jours d'emprunt  
        $emprunt_query = "INSERT INTO emprunts (user_id, media_id, date_emprunt, date_retour_prevue, statut)  
        | VALUES (?, ?, NOW(), ?, 'En cours')";  
        $emprunt_success = db_execute($emprunt_query, [$user_id, $media_id, $date_retour_prevue]);  
        $stock_success = decrement_stock($media_id);  
  
        if ($emprunt_success && $stock_success) {  
            db_execute("COMMIT");  
            return true;  
        } else {  
            db_execute("ROLLBACK");  
            return false;  
        }  
    } catch (Exception $e) {  
        db_execute("ROLLBACK");  
        return false;  
    }  
}
```

Si l'une des requêtes échoue ou relève une erreur, le script exécute un *ROLLBACK* qui annule tout ce qui a commencé à être écrit. Cette précaution permet de maintenir une base cohérente.

Afin de prémunir l'application contre le risque de failles **XSS**, l'ensemble des données saisies par les utilisateurs est systématiquement assaini avant insertion dans les vues à l'aide d'un helper d'échappement centralisé s'appuyant sur la fonction `htmlspecialchars` avec l'option `ENT_QUOTES`.

```
/* Sécurise l'affichage d'une chaîne de caractères (protection XSS)
*/
function escape($string)
{
    if ($string === null) {
        $string = '';
    }
    return htmlspecialchars((string)$string, ENT_QUOTES, 'UTF-8');
}
```

Déploiement et configuration du serveur

Une fois l'application stable, j'ai procédé à son déploiement en ligne sur un serveur privé virtuel (**VPS**) sous **Ubuntu**, administré à l'aide de l'interface **Plesk Obsidian**. Pour rapatrier notre code proprement, j'ai connecté notre dépôt GitHub directement dans Plesk. Cela permet de cloner l'application dans le répertoire public `/httpdocs/mediatheque` et de faire nos mises à jour en un clic via un simple bouton de synchronisation (*Git Pull*).

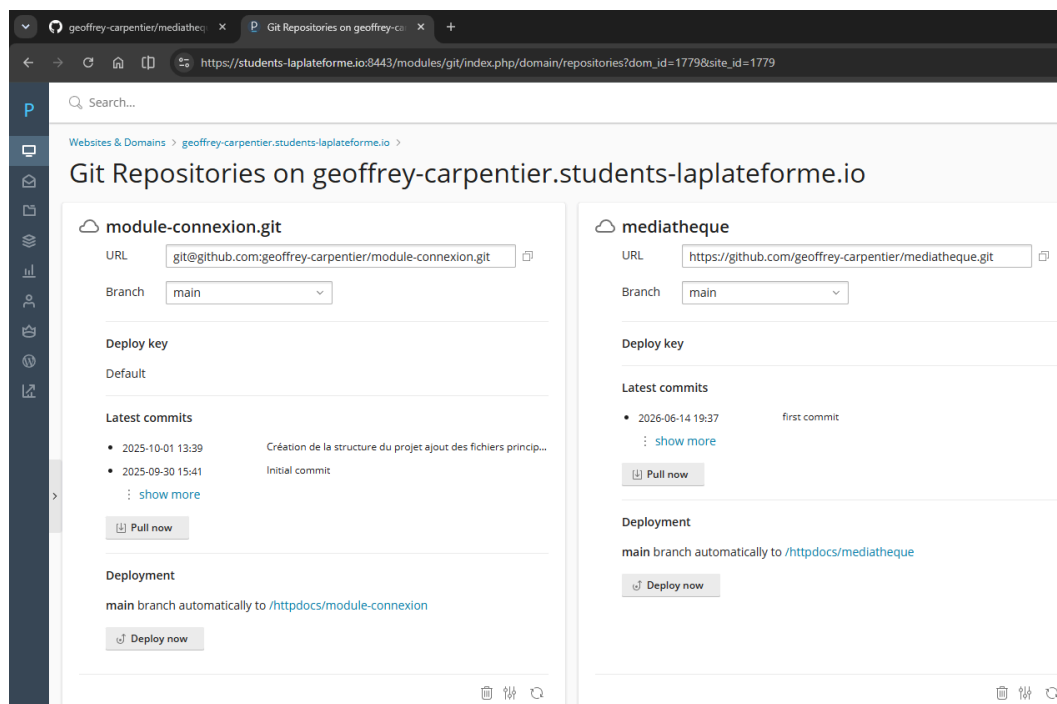


Figure 6.1 - Suivi Git et historique des versions en production sous Plesk.

Concernant la base de données, j'ai créé un utilisateur MySQL de production propre à l'application. Je lui ai attribué un mot de passe fort et des droits limités à cette seule base, avant d'importer la structure de nos tables avec phpMyAdmin.

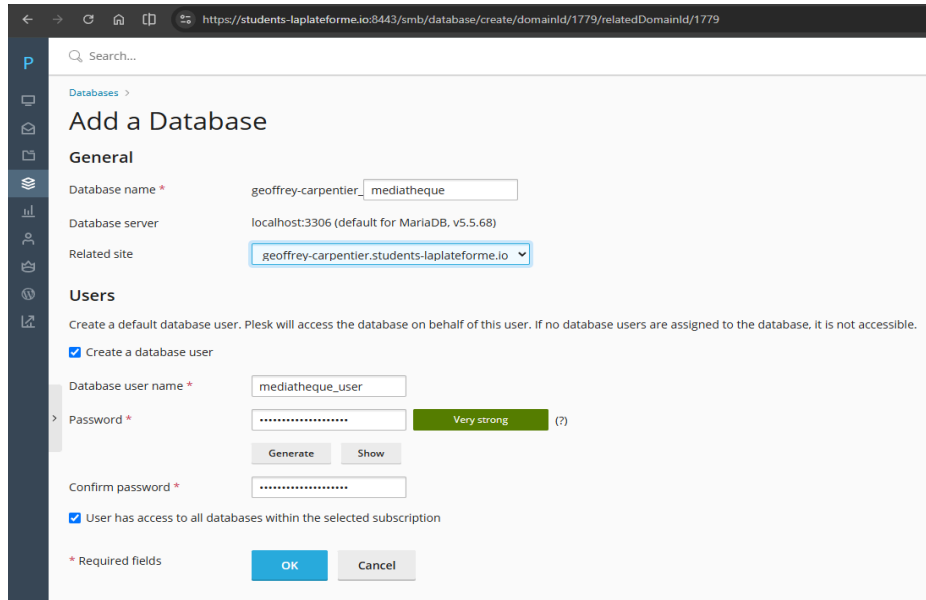


Figure 6.2 - Création base de données + utilisateur MySQL (Plesk)

Pour lier le code à ce nouvel environnement, j'ai créé un fichier d'environnement **.env** sur le serveur (non partagé sur Git) pour y cacher nos identifiants de base de données. J'ai configuré `config/database.php` pour lire ces variables et j'ai modifié notre variable `BASE_URL` pour qu'elle pointe vers notre nom de domaine. Enfin, j'ai ajusté le fichier **.htaccess** pour gérer la réécriture d'URL afin que les visiteurs arrivent directement sur l'application en masquant proprement le dossier `./public` de notre architecture.

```
// Adresse de l'hôte MySQL (généralement 'localhost')
define('DB_HOST', 'localhost');
// Nom de la BDD (doit correspondre au nom donné lors de sa génération)
define('DB_NAME', 'geoffrey-carpentier_mediatheque');
// Nom d'utilisateur pour la connexion MySQL (permissions SELECT, INSERT, UPDATE, DELETE requises)
define('DB_USER', 'mediatheque_user');
// Mot de passe de l'utilisateur MySQL (si pas de mot de passe => laisser vide '')
define('DB_PASS', '*****');
// Jeu de caractères utilisé pour la BDD ('utf8mb4' recommandé)
define('DB_CHARSET', 'utf8mb4');
// URL de base de l'application (ajuster selon le nom du dossier)
define('BASE_URL', 'https://geoffrey-carpentier.students-laplateforme.io/mediatheque/public');
```

Figure 6.3 - Extrait du fichier d config/database.php (Plesk)

SSL/TLS Certificate for geoffrey-carpentier.students-laplateforme.io

Let's Encrypt Entry-level protection
[Choose another]

Let's Encrypt is a certificate authority (CA) that allows you to create a free SSL/TLS certificate for your domain. By proceeding you acknowledge that you have read and agree to the [Let's Encrypt Terms of Service](#).
Note: The certificate will be automatically renewed 30 days in advance before its expiration.

Email address *

Make sure to use a valid email address to receive important notifications and warnings.

☒ Secure the domain name
geoffrey-carpentier.students-laplateforme.io

☐ Secure the wildcard domain (including www and webmail)
*.geoffrey-carpentier.students-laplateforme.io

☐ Include a "www" subdomain for the domain and each selected alias
www.geoffrey-carpentier.students-laplateforme.io

☐ Secure webmail on this domain
The mail service is not active for the domain. [Configure mail settings](#)

Passage au HTTPS : J'ai généré un certificat de chiffrement gratuit via **Let's Encrypt** et configuré une redirection automatique pour que toutes les requêtes en HTTP classique basculent sur le protocole sécurisé HTTPS.

Figure 6.4 - Création du certificat SSL avec Let's Encrypt

Permissions des dossiers (CHMOD) :

Les droits d'écriture sur le dossier d'envoi des fichiers (*public/uploads/covers*) ont dû être adaptés. Seul le groupe du serveur web Plesk (*psac/n*) y est autorisé pour des raisons de sécurité, et j'ai bloqué l'affichage de l'arborescence des dossiers (Options -Indexes).

Directory permissions

covers

	Read	Write	Execute/search
Owner	☒	☒	☒
Group	☒	☒	☒
Others	☒	☐	☒

☒ Change permissions recursively

Figure 6.5- Permissions CHMOD du dossier d'upload

Le premier déploiement sur le serveur distant a mis en évidence deux blocages inattendus.

D'une part, le serveur de production exécutait une version de *PHP* antérieure à celle de mon poste local, provoquant des erreurs de syntaxe sur les fonctionnalités récentes.

D'autre part, les règles de réécriture d'URL nécessaires au routage de l'application n'étaient pas prises en compte. J'ai résolu ces problèmes en mettant à jour la configuration d'exécution de l'hébergeur et en ajustant le fichier **.htaccess** :

```
function dispatch() {  
    // Parser l'URL pour extraire les composants de routage  
    $route = parse_request_url();  
  
    $controller_name = $route['controller'];  
    $action_name = $route['action'];  
    $params = $route['params'];  
  
    // --- DÉBUT CORRECTION ---  
    // Gérer l'alias : si l'URL demande 'medias', on utilise le contrôleur 'media'  
    if ($controller_name === 'medias') {  
        $controller_name = 'media';  
    }  
    // --- FIN CORRECTION ---  
}
```

Figure 6.6 - Correctif du routeur en production.

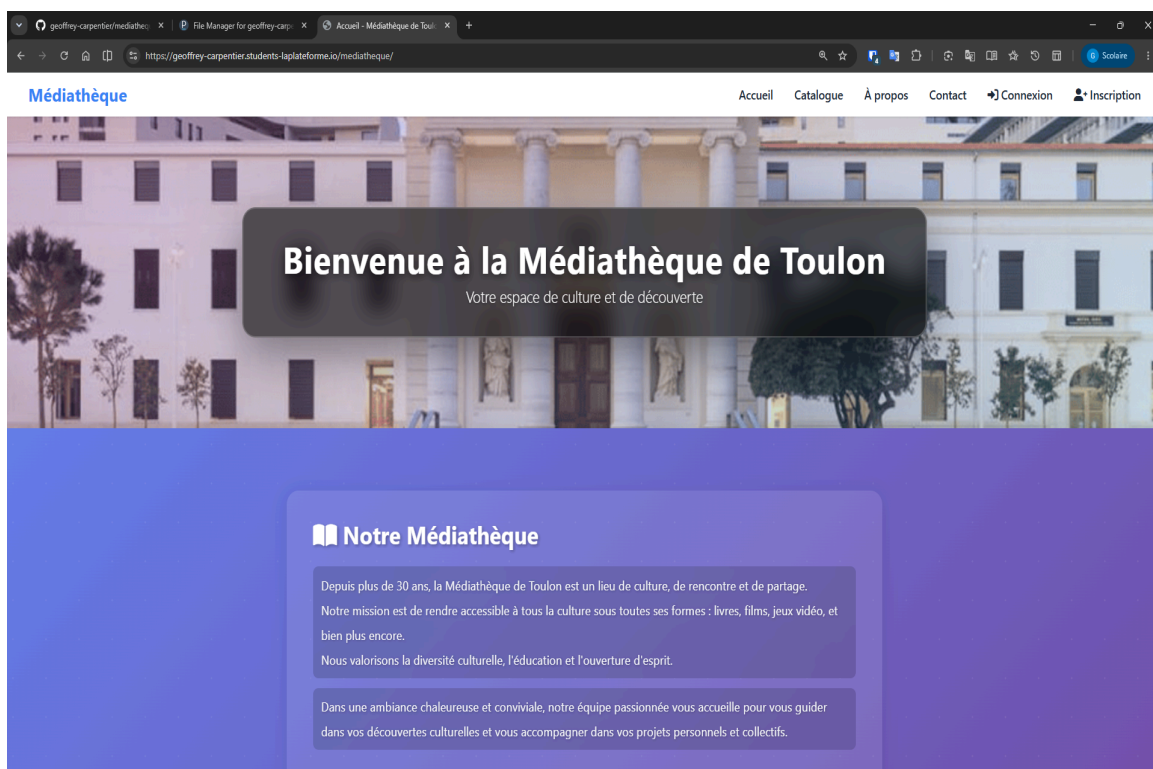


Figure 6.7 - Site Médiathèque en production, HTTPS actif.

DOSSIER PROFESSIONNEL ^(DP)

2. Précisez les moyens utilisés :

- **Matériels et logiciels** : poste sous Windows 11, IDE Visual Studio Code, VPS Ubuntu contrôlé par Plesk Obsidian, client SSH, phpMyAdmin, Laragon (développement local).
- **Technologies et configurations** : Environnement **PHP 8.1** (extension **PDO** et transactions atomiques), serveur **Apache** (réécriture d'URL via `mod_rewrite` et `.htaccess`), base **MySQL 8.0** et interface **phpMyAdmin**, Sécurisation par **HTTPS** via **Let's Encrypt**, gestion des **permissions CHMOD** sur Ubuntu, et déploiement continu avec **Git/GitHub**.

3. Avec qui avez-vous travaillé ?

J'ai collaboré au développement des composants d'accès aux données dans une équipe de quatre développeurs, puis j'ai conduit en autonomie le déploiement sur Plesk, les accès VPS étant individuels.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ La Plateforme (Toulon)

Chantier, atelier, service ▶ projet collaboratif « Médiathèque

Période d'exercice ▶ Du : 06/10/2025 au : 03/11/2025

5. Informations complémentaires *(facultatif)*

Cette expérience de mise en production s'est avérée particulièrement formatrice. Elle m'a permis de me confronter aux réalités d'un vrai serveur, bien plus exigeantes sur l'aspect sécuritaire que dans les conditions qu'un environnement de développement local comme **Laragon** peut offrir.

L'investigation des blocages liés à la version **PHP** et la reconfiguration des règles du routeur m'ont donné une occasion de diagnostiquer des pannes système puis de surmonter ces contraintes.

Dans une optique d'amélioration, pour mes futurs déploiements j'envisage d'adopter des solutions de conteneurisation **Docker**. En effet, l'encapsulation de ma stack technique garantirait une correspondance totale entre mon poste de travail et le serveur distant, avec pour impact l'élimination des dysfonctionnements liés aux différences d'environnement lors de la mise en ligne, ce qui représente une piste prometteuse pour contourner les difficultés énoncées plus haut.

Hébergement Plesk: <https://geoffrey-carpentier.students-laplateforme.io/mediatheque/>

Dépôt GitHub du projet : <https://github.com/geoffrey-carpentier/mediatheque>

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Non renseigné (rubrique facultative).		

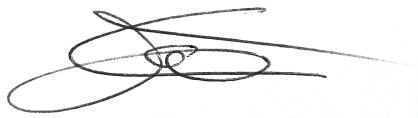
Déclaration sur l'honneur

Je soussigné **Geoffrey CARPENTIER**
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts
et que je suis l'auteur des réalisations jointes.

Fait à**TOULON**..... le**10/07/2026**.....

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Illustrations intégrées aux exemples de pratique professionnelle.

ANNEXES

(Si le RC le prévoit)

Aucune annexe complémentaire. Les illustrations figurent dans les exemples.